

ML Specification Report — spaceship-titanic

Passenger Transportation Prediction • AIDE agent (20-step tree search, Claude Sonnet via shim)

Scores grounded exclusively in `facts_aide.json` for run `2-ambrosial-misty-adder`; pipeline configuration read from AIDE's own `best_solution.py` and `report.md`, step-level intent from `journal.json`, and task metadata from `competitions/spaceship-titanic/config.yaml`.

Contents	
Passenger Transportation Prediction • AIDE agent (20-step tree search, Claude Sonnet via shim)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	4
The search trajectory	4
Training configuration	6
Inference procedures	7
Evaluation & Benchmarking	7

Overview

The task is to predict, for each of 4,277 test passengers, whether they were transported to an alternate dimension — a nearly balanced binary classification scored by plain **accuracy**. This run was executed by **AIDE**, a tree-search coding agent: each step writes one complete `solution.py`, executes it in a sandbox under a 30-minute wall clock, reads back the printed metric or the traceback, and mutates the incumbent. Over **20 steps** it produced **19 scored solutions and 1 crash**, and its champion — found at **step 4, the fifth solution it ever wrote** — is a **CatBoost classifier whose only distinguishing feature is a predictively imputed `cryoSleep` probability**, scoring **5-fold CV accuracy 0.8156** locally and **0.805** on the public leaderboard, for **rank 553 / 1755 (68.5th percentile)**. The score table records AIDE as the winner of this competition in both comparisons (`local_winner: aide`, `lb_winner: aide (pub)`).

The defining characteristic of this run is that **AIDE found its best solution 20% of the way through the budget and then failed to improve on it for fifteen consecutive steps**. Step 19 tied 0.8156 exactly; nothing beat it. This is the cleanest example in the batch of a tree search that converges early and then spends its remaining budget confirming its own ceiling.

Why it matters. Spaceship Titanic is the canonical modern beginner tabular benchmark — small (8,693 training rows), missing-value-rich, and full of latent structure hidden inside composite string fields

(`PassengerId` encodes travelling groups, `Cabin` encodes deck/number/side). It rewards data understanding over compute, which makes it a sharp test of whether an autonomous agent can reason about a dataset rather than merely throw model families at it.

Data

Purpose of Data. Predict the boolean `Transported` outcome per passenger. **Data Format** is **tabular CSV** with mixed types — numeric, categorical, boolean, and two composite string keys. **Data Volume**, per `config.yaml`, is **8,693 training rows / 4,277 test rows**; three orders of magnitude smaller than the Playground Series episodes in this batch, which is why the entire 20-step search cost under five minutes of execution time.

Data Quality is the substance of this competition, and AIDE engaged with it seriously — but implicitly, through code, never through analysis. There is no EDA artifact in the run because the agent has no EDA stage; what exists is a sequence of imputation strategies, each an implicit hypothesis about the missingness mechanism. `config.yaml` records missing values in most columns at roughly 2%, a near-balanced target (50.4% transported), and the two structural encodings (`PassengerId` = `GGGG_PP`, `Cabin` = `deck/num/side`). AIDE independently recovered both structures at step 0 and never lost them.

The **defining wrinkle** is `CryoSleep`, and AIDE's handling of it is the single interesting result of the run. The obvious heuristic — a passenger with zero total spending must have been in cryosleep — is what AIDE wrote at steps 0 and 1. At step 4 it replaced that hard rule with **predictive imputation**: train an auxiliary CatBoost classifier on the rows where `CryoSleep` is observed, then emit a continuous `CryoSleepProb` feature that equals 0 or 1 where the value is known and equals the model's predicted probability where it is missing. That one change moved the score from 0.81157 to 0.8156, the largest gain in the run, and AIDE's own report reads the result correctly: the zero-spend heuristic was discarding signal that a model trained on the full feature set could recover, and preserving uncertainty as a probability beat committing to a hard fill.

What is equally instructive is that **the same idea did not generalize**. AIDE extended predictive imputation to `HomePlanet / Destination` at step 9 (0.81376) and to `Age` at step 16 (0.8118); both underperformed the `CryoSleep`-only version. Group-mode deterministic imputation at step 10 scored 0.81054. The agent tested its own best idea four more ways and each extension was worse — an honest, well-executed ablation of a single axis, and the strongest piece of experimental design anywhere in this batch.

Annotation Guidelines. The label is `Transported` $\in \{\text{False}, \text{True}\}$, cast to `int` for training. The submission is a **hard boolean label** per `PassengerId`, so accuracy is computed on a thresholded decision, not a ranking.

Feature Set. The champion feeds the model 16 columns, all derived from the raw file — no external data, no polynomial expansion:

Group	Features
Raw numeric (6)	Age , RoomService , FoodCourt , ShoppingMall , Spa , VRDeck
PassengerId decomposition	GroupSize (count of passengers sharing a travelling group)
Cabin decomposition	Deck , CabinNum (numeric), Side
Spending aggregate	TotalSpend (sum of the five spend columns)
Raw categorical	HomePlanet , Destination , VIP
Predictively imputed	CryoSleepProb (continuous), CryoSleep (kept as a categorical with an explicit "Missing" level)

Worth noting for anyone rebuilding this pipeline: the champion's `feature_engineer` function also computes `Group` , `GroupNumber` and `NoSpend` , but **none of the three appears in `feature_cols`** — they are calculated on every run and then silently discarded. Only `GroupSize` , which is derived from `Group` , survives into the model. This is dead code carried forward from an earlier node, and it means the group-identity and zero-spend signals AIDE explicitly reasoned about at step 0 are not actually in the winning feature set.

Numeric missing values are filled with the **training-set median**, computed once on `train` and applied to both splits; categorical missing values become an explicit `"Missing"` category rather than being imputed, which is CatBoost's preferred treatment. `Name` is **not** used in the champion: AIDE extracted surname/family-size features at step 12 and scored 0.81134, below the incumbent, so the column was dropped.

Splitting strategy. `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` , fixed from step 0 and never varied, so all 19 scored nodes are directly comparable. With only 8,693 rows each fold holds about one fifth of the training set for validation, and AIDE's own report repeatedly flags wide per-fold accuracy variance as a source of noise — a warning it acted on twice (seed bagging at step 17, repeated-CV reasoning in several plans) without ever getting a gain from it. The competition has a rolling public leaderboard and no private split, so `facts_aide.json` carries a **public LB score only**.

Models & Architecture

Purpose of Architecture. A binary classifier maximizing accuracy on a near-balanced target.

Architecture Type is a **single CatBoost gradient-boosted decision-tree classifier**, preceded by a **second, auxiliary CatBoost classifier used purely as an imputer**. There is no ensemble in the champion: AIDE tried averaging CatBoost with XGBoost at step 14 (0.81284) and 3-seed CatBoost bagging at step 17 (0.81065), and both lost to the single model.

Input Format is a 16-column mixed matrix — 10 numeric columns (including the continuous `CryoSleepProb`) and 6 categorical columns declared to CatBoost by positional index; **Input Dimension**

is 16, a 1-D vector with no spatial or sequential structure. The auxiliary imputer sees a 14-column subset (the same features minus `CryoSleepProb` and `CryoSleep` itself).

Architecture Description. The **auxiliary imputer** is `CatBoostClassifier(iterations=300, depth=5, learning_rate=0.08, loss_function="Logloss", random_seed=42)`, fitted once on the rows where `CryoSleep` is observed and then applied to all rows of both splits; its output is blended into `CryoSleepProb` as `where(observed, CryoSleep.astype(float), predicted_probability)`. The **main classifier** is `CatBoostClassifier(iterations=500, depth=6, learning_rate=0.05, loss_function="Logloss", eval_metric="Accuracy", random_seed=42, early_stopping_rounds=50)` with `use_best_model=True` inside each CV fold. **Model Complexity** follows from CatBoost's symmetric (oblivious) trees:

```
depth 6 → 2^6 = 64 leaves per tree
up to 500 trees per fold model, truncated by early stopping
```

so a few tens of thousands of leaves at most — a deliberately small model for a small dataset. Note one structural detail with real consequences: the fold models accumulate a `test_preds` vector during CV, but the submitted predictions do **not** come from it. The champion refits a **final model on the full training set with early stopping removed** and predicts from that single model, discarding the 5-fold test ensemble it had already computed.

Training procedures

The search trajectory

Of the **20 steps** in the budget, **19 produced a score and 1 crashed**:

```
20 - 19 = 1 buggy step
```

The single failure is a `TypeError` (`buggy_exc_types: {"TypeError": 1}`) at **step 6** — the one step absent from the metric trajectory. Its cause is precise and AIDE identified it exactly: XGBoost's native categorical path (`enable_categorical=True`) cannot convert `numpy.bool_` category values, which is what `CryoSleep` and `VIP` become when read from this file. Step 8 fixed it by encoding those two columns as strings before handing them to XGBoost, and the repaired run scored 0.8133 — the best non-CatBoost result of the entire search. This was a genuinely well-diagnosed bug, repaired in one step with no collateral damage.

Compute was never the constraint here. The whole 20-step search consumed

```
289.4 s total → 289.4 / 60 ≈ 4.82 minutes
mean 14.5 s per node, max 57.2 s
```

Against a 1,800 s harness wall, the slowest node used 57.2 s — **there were no timeouts, and the budget was never binding**. That makes this run the cleanest read on AIDE's search behaviour

uncontaminated by resource failures: whatever it failed to find, it failed to find for want of ideas, not for want of compute.

Step	What AIDE changed	CV accuracy
0	LightGBM, one-hot encoding, group-median imputation (baseline)	0.81008
1	CatBoost, native categoricals, "Missing" levels, zero-spend CryoSleep rule	0.81157
2	group/family-mode imputation + surname-based Age fill	0.80962
3	per-group spend ratios, per-column zero-spend flags, spend $\times$ age	0.81042
4	auxiliary-CatBoost predictive imputation of CryoSleep $\rightarrow$ CryoSleepProb	0.8156
5	Random Forest, sklearn pipeline	0.80593
6	XGBoost with native categoricals	TypeError (numpy.bool_ categories)
7	PyTorch MLP, two hidden layers, 5-fold ensemble	0.81054
8	bug fix: string-encode boolean categoricals $\rightarrow$ XGBoost runs	0.8133
9	extend predictive imputation to HomePlanet / Destination	0.81376
10	revert to CryoSleep-only + group-mode fills for 4 categoricals	0.81054
11	+ GroupCryoSleepProb group-mean aggregate	0.81468
12	+ Surname categorical and FamilySize	0.81134
13	CatBoost hyperparameter tuning (lower LR, more iterations, L2)	0.81042
14	CatBoost + XGBoost probability averaging	0.81284
15	+ GroupDeck , GroupHomePlanet , IsSolo flag	0.81146
16	extend predictive imputation to Age (auxiliary regressor)	0.8118
17	3-seed CatBoost bagging per fold	0.81065
18	+ CabinRegion binning of CabinNum	0.81169
19	OOF decision-threshold search	0.8156

The full search band is

$$0.8156 - 0.80593 = 0.00967$$

and the total gain over AIDE's own first solution is

$$0.8156 - 0.81008 = 0.00552$$

All of it was banked by step 4. The fifteen steps that followed — five feature-engineering variants, three alternative model families, an ensemble, seed bagging, hyperparameter tuning, three further

imputation strategies, and a threshold search — produced not one improvement. The closest any of them came was step 11's group-level `CryoSleepProb` aggregate:

```
0.8156 - 0.81468 = 0.00092 (best challenger, step 11)
```

Step 19's threshold search is the fitting end to the run: it swept the OOF decision threshold, concluded that 0.5 was already optimal, and returned **exactly** 0.8156 — the incumbent's score, unchanged to the last recorded digit. The search's last act was to prove it had nothing left to give.

Two observations about how AIDE searched are worth recording. First, it is **strongly anchored to its incumbent**: steps 9 through 18 are ten consecutive plans that open by naming the champion's score ("the best-performing pipeline (0.8156, CryoSleep-only predictive imputation)"), and each proposes a single atomic delta on top of it. That discipline is the reason the run never regressed catastrophically — but it also means the search became a one-step-lookahead neighbourhood scan around a local optimum it had found by luck at step 4. Second, AIDE **never attempted stacking**, which its own report names as the top future-work item; with four working base model families in hand (CatBoost 0.8156, XGBoost 0.8133, MLP 0.81054, Random Forest 0.80593) and 15 idle steps, a meta-learner on OOF predictions was both obvious and affordable. The search simply never proposed it.

Training configuration

The **Loss Function** is binary log-loss (`loss_function="Logloss"`) for both the main model and the auxiliary imputer, with **model selection and early stopping on accuracy** (`eval_metric="Accuracy"`) — correctly matched to the competition metric rather than to the training objective. No class weighting is applied, which is right for a near-balanced target; AIDE considered adding `class_weights` at step 13 and that node scored 0.81042, below the incumbent. The **Optimization Algorithm** is gradient boosting with CatBoost's ordered boosting and symmetric trees — **not** SGD/ADAM. The **Learning Rate** is the boosting shrinkage: 0.05 for the main model, 0.08 for the auxiliary imputer; step 13's attempt to lower it and compensate with more iterations made things worse. There is **no Learning Rate Scheduler** (N/A — GBDT convergence is governed by per-fold early stopping at 50 rounds, not a schedule) and **no Batch Size** (N/A — full-dataset boosting, not mini-batched).

Training Duration for the whole search was **289.4 s** over 20 nodes, mean **14.5 s**, max **57.2 s** — the cheapest run in this batch by two orders of magnitude. Per-node wall time for the champion specifically is not carried in `facts_aide.json` — **not recorded**. **Training Memory** was never instrumented — **not recorded**; an $8,693 \times 16$ frame is trivial.

Transfer Learning is N/A (no pretrained weights). AIDE has **no cross-competition memory** — every run starts cold — and its only analogue is the within-run journal, which here is unusually visible: nine plans quote the incumbent's score directly, and step 6's plan explicitly reasons about which model families are "not yet tried in memory". The nearest thing to transfer inside the run is the **auxiliary imputer**, a model trained on one target (`CryoSleep`) whose output becomes a feature for another (`Transported`) — a small, genuine instance of learned-representation reuse. **Data Augmentation** is N/A (tabular); the

analogues attempted were feature engineering (group aggregates, family features, cabin binning — all rejected) and seed bagging (step 17, rejected).

Reproducibility Standards are simple and adequate: a single `random_state=42` on the fold split and `random_seed=42` on both CatBoost models, applied consistently. CatBoost on a single machine with fixed seed is reproducible in practice, though no determinism flags or thread pinning were set, so bit-identical results across differing hardware are not guaranteed. There is **no rebuild or reproduction gate** — AIDE never retrains its champion to verify the cached metric — and the submitted artifact is simply the `working/submission.csv` written by the winning node.

Inference procedures

Decision Threshold is the substantive inference decision here, because the metric is accuracy on hard labels. The champion applies **0.5** (`final_preds = final_preds_proba >= 0.5`), and this is not an unexamined default: step 19 explicitly collected out-of-fold probabilities across all five folds and searched for the accuracy-maximizing cut, returning 0.8156 — identical to the incumbent — which confirms 0.5 was already optimal and that the model's probabilities are well calibrated for this target's near-balanced prior.

Post-processing is a cast to `bool` and a right-join onto `sample_submission`'s `PassengerId` column to guarantee row order and completeness. The important subtlety, noted above, is that the **shipped predictions come from a single model refit on all 8,693 training rows with early stopping removed**, not from the 5-fold ensemble whose accuracy is reported — the fold-averaged `test_preds` vector is computed and then discarded. The reported 0.8156 therefore describes a slightly different estimator from the one that produced the submission. **Inference Duration** and **Inference Memory** were not profiled — **not recorded**.

Evaluation & Benchmarking

Task for Performance Evaluation. Predict a boolean `Transported` label for each of the 4,277 test passengers, scored by accuracy.

Performance Metrics. AIDE's champion scores **5-fold CV accuracy 0.8156** locally and **0.805** on the public leaderboard, for **rank 553 / 1755** = the **68.5th percentile**. The gap between the two is the largest relative CV-to-LB drop in this batch:

```
0.8156 - 0.805 = 0.0106
0.0106 × 4277 ≈ 45.3 test rows
```

Roughly forty-five passengers' worth of optimism. Part of this is ordinary small-sample noise — 4,277 test rows put the leaderboard's own standard error in the same range — but part of it is structural: the reported 0.8156 was selected as the maximum over nineteen scored candidates on a single fixed 5-fold

split holding only a fifth of 8,693 rows out per fold, so the winner's-curse component is real. The competition has no private leaderboard, so this cannot be checked against a held-out split.

The gain the search actually delivered over its own starting point is

0.8156 - 0.81008 = 0.00552

all of it from a single idea at step 4, with the remaining fifteen steps contributing zero.

Performance Benchmarking. `facts_aide.json` 's score table records the verdict fields `local_winner: aide` and `lb_winner: aide (pub)` — AIDE won this competition against both comparator lanes — but it does **not** carry `mine_priv` or `nvidia_priv` values for spaceship-titanic, so a numeric three-way table cannot be constructed and none is invented here:

Agent	Approach	Public LB accuracy	Note
AIDE	20-step search → CatBoost + predictive <code>CryoSleep</code> imputation	0.805	rank 553 / 1755, 68.5th pct; recorded winner
From-scratch agent	not carried in <code>facts_aide.json</code>	—	recorded as losing both comparisons
NVIDIA	not carried in <code>facts_aide.json</code>	—	no figure available

The honest summary is a **narrow win purchased by one good idea in under five minutes of compute**. AIDE's strength on this competition was reasoning about the missingness mechanism — recognizing that a hard zero-spend rule for `CryoSleep` destroys information and replacing it with a calibrated probability — and it then validated that idea properly by failing to extend it four different ways. Its weakness is equally clear: having found the answer at step 4, it had no mechanism to recognize convergence, no willingness to abandon its incumbent for a structurally different approach, and it left the one obviously promising avenue (stacking its four working base models) untried across fifteen free steps and a 30-minute compute box whose slowest node used 57.2 s.